

JOBSHEET 14
Tree



Disusun oleh:

Nama : Galih Candra Kirana
No. Absen : 10
Kelas : TI 1G
NIM : 254107020080

POLITEKNIK NEGERI MALANG
Jl. Soekarno Hatta No.9, Jatimulyo, Kec. Lowokwaru, Kota Malang, Jawa Timur 65141
TAHUN 2025-2026

14.1 Tujuan Praktikum

Tujuan Praktikum Setelah melakukan praktikum ini, mahasiswa mampu:

1. memahami model Tree khususnya Binary Tree
2. membuat dan mendeklarasikan struktur algoritma Binary Tree.
3. menerapkan dan mengimplementasikan algoritma Binary Tree dalam kasus Binary Search Tree

14.2 Percobaan 1: Implementasi Binary Search Tree menggunakan Linked List

14.2.1 Langkah-langkah Percobaan

Class Mahasiswa10:

```
package Jobsheet14;

public class Mahasiswa10 {
    String nim;
    String nama;
    String kelas;
    double ipk;

    Mahasiswa10() {
    }

    Mahasiswa10(String nim, String nama, String kelas, double ipk) {
        this.nim = nim;
        this.nama = nama;
        this.kelas = kelas;
        this.ipk = ipk;
    }

    public void tampilInformasi() {
        System.out.println("NIM: " + nim + " " +
            "Nama: " + nama + " " +
            "Kelas: " + kelas + " " +
            "IPK: " + ipk);
    }
}
```

Class Node10:

```
package Jobsheet14;

public class Node10 {
    Mahasiswa10 mahasiswa;
    Node10 left, right;
```

```

public Node10() {
}

public Node10(Mahasiswa10 mahasiswa) {
    this.mahasiswa = mahasiswa;
    left = right = null;
}
}

```

Class BinaryTree10:

```

package Jobsheet14;

public class BinaryTree10 {
    Node10 root;

    public BinaryTree10() {
        root = null;
    }

    public boolean isEmpty() {
        return root == null;
    }

    public void add(Mahasiswa10 mahasiswa) {
        Node10 newNode = new Node10(mahasiswa);
        if (isEmpty()) {
            root = newNode;
        } else {
            Node10 current = root;
            Node10 parent = null;
            while (true) {
                parent = current;

```

```

        if (mahasiswa.ipk < current.mahasiswa.ipk) {
            current = current.left;
            if (current == null) {
                parent.left = newNode;
                return;
            }
        } else {
            current = current.right;
            if (current == null) {
                parent.right = newNode;
                return;
            }
        }
    }
}

```

```

public boolean find(double ipk) {
    boolean result = false;
    Node10 current = root;
    while (current != null) {
        if (current.mahasiswa.ipk == ipk) {
            result = true;
            break;
        } else if (ipk > current.mahasiswa.ipk) {
            current = current.right;
        } else {
            current = current.left;
        }
    }
    return result;
}

```

```
public void traversePreOrder(Node10 node) {  
    if (node != null) {  
        node.mahasiswa.tampilInformasi();  
        traversePreOrder(node.left);  
        traversePreOrder(node.right);  
    }  
}
```

```
public void traverseInOrder(Node10 node) {  
    if (node != null) {  
        traverseInOrder(node.left);  
        node.mahasiswa.tampilInformasi();  
        traverseInOrder(node.right);  
    }  
}
```

```
public void traversePostOrder(Node10 node) {  
    if (node != null) {  
        traversePostOrder(node.left);  
        traversePostOrder(node.right);  
        node.mahasiswa.tampilInformasi();  
    }  
}
```

```
public Node10 getSuccessor (Node10 del) {  
    Node10 successor = del.right;  
    Node10 successorParent = del;  
    while (successor.left != null) {  
        successorParent = successor;  
        successor = successor.left;  
    }  
}
```

```

    if (successor != del.right) {
        successorParent.left = successor.right;
        successor.right = del.right;
    }
    return successor;
}

public void delete (double ipk) {
    if (isEmpty()) {
        System.out.println("Binary tree kosong");
        return;
    }
    // cari node (current) yang akan dihapus
    Node10 parent = root;
    Node10 current = root;
    boolean isLeftChild = false;
    while (current != null) {
        if (current.mahasiswa.ipk == ipk) {
            break;
        } else if (ipk < current.mahasiswa.ipk) {
            parent = current;
            current = current.left;
            isLeftChild = true;
        } else if (ipk > current.mahasiswa.ipk) {
            parent = current;
            current = current.right;
            isLeftChild = false;
        }
    }

    // penghapusan
    if (current == null) {

```

```

        System.out.println("Data tidak ditemukan");
        return;
    } else {
        // jika tidak ada anak (leaf), maka node dihapus
        if (current.left == null && current.right == null) {
            if (current == root) {
                root = null;
            } else {
                if (isLeftChild) {
                    parent.left = null;
                } else {
                    parent.right = null;
                }
            }
        }
        } else if (current.left == null) { // jika hanya punya 1 anak (kanan)
            if (current == root) {
                root = current.right;
            } else {
                if (isLeftChild) {
                    parent.left = current.right;
                } else {
                    parent.right = current.right;
                }
            }
        }
        } else if (current.right == null) { // jika hanya punya 1 anak (kiri)
            if (current == root) {
                root = current.left;
            } else {
                if (isLeftChild) {
                    parent.left = current.left;
                } else {
                    parent.right = current.left;
                }
            }
        }
    }
}

```

```

        }
    }
    } else { // jika punya 2 anak
        Node10 successor = getSuccessor(current);
        System.out.println("Jika 2 anak, current = ");
        successor.mahasiswa.tampilInformasi();
        if (current == root) {
            root = successor;
        } else {
            if (isLeftChild) {
                parent.left = successor;
            } else {
                parent.right = successor;
            }
        }
        successor.left = current.left;
    }
}
}
}
}

```

Class BinaryTreeMain10:

```

package Jobsheet14;

public class BinaryTreeMain10 {

    public static void main(String[] args) {
        BinaryTree10 bst = new BinaryTree10();

        bst.add(new Mahasiswa10("244160121", "Ali", "A", 3.57));
        bst.add(new Mahasiswa10("244160221", "Badar", "B", 3.85));
        bst.add(new Mahasiswa10("244160185", "Candra", "C", 3.21));
    }
}

```

```
bst.add(new Mahasiswa10("244160220", "Dewi", "B", 3.54));
```

```
System.out.println("\nDaftar semua mahasiswa (in order traversal):");
```

```
bst.traverseInOrder(bst.root);
```

```
System.out.println("\nPencarian data mahasiswa:");
```

```
System.out.print("Cari mahasiswa dengan ipk: 3.54 : ");
```

```
String hasilCari = bst.find(3.54) ? "Ditemukan" : "Tidak ditemukan";
```

```
System.out.println(hasilCari);
```

```
System.out.print("Cari mahasiswa dengan ipk: 3.22 : ");
```

```
hasilCari = bst.find(3.22) ? "Ditemukan" : "Tidak ditemukan";
```

```
System.out.println(hasilCari);
```

```
bst.add(new Mahasiswa10("244160131", "Devi", "A", 3.72));
```

```
bst.add(new Mahasiswa10("244160205", "Ehsan", "D", 3.37));
```

```
bst.add(new Mahasiswa10("244160170", "Fizi", "B", 3.46));
```

```
System.out.println("\nDaftar semua mahasiswa setelah penambahan 3  
mahasiswa:");
```

```
System.out.println("InOrder Traversal:");
```

```
bst.traverseInOrder(bst.root);
```

```
System.out.println("\nPreOrder Traversal:");
```

```
bst.traversePreOrder(bst.root);
```

```
System.out.println("\nPostOrder Traversal:");
```

```
bst.traversePostOrder(bst.root);
```

```
System.out.println("\nPenghapusan data mahasiswa");
```

```
bst.delete(3.57);
```

```

        System.out.println("\nDaftar semua mahasiswa setelah penghapusan 1
mahasiswa (in order traversal):");

        bst.traverseInOrder(bst.root);    }

    }

```

14.2.2 Verifikasi Hasil Percobaan

```

Daftar semua mahasiswa (in order traversal):
NIM: 244160185 Nama: Candra Kelas: C IPK: 3.21
NIM: 244160220 Nama: Dewi Kelas: B IPK: 3.54
NIM: 244160121 Nama: Ali Kelas: A IPK: 3.57
NIM: 244160221 Nama: Badar Kelas: B IPK: 3.85

Pencarian data mahasiswa:
Cari mahasiswa dengan ipk: 3.54 : Ditemukan
Cari mahasiswa dengan ipk: 3.22 : Tidak ditemukan

Daftar semua mahasiswa setelah penambahan 3 mahasiswa:

InOrder Traversal:
NIM: 244160185 Nama: Candra Kelas: C IPK: 3.21
NIM: 244160205 Nama: Ehsan Kelas: D IPK: 3.37
NIM: 244160170 Nama: Fizi Kelas: B IPK: 3.46
NIM: 244160220 Nama: Dewi Kelas: B IPK: 3.54
NIM: 244160121 Nama: Ali Kelas: A IPK: 3.57
NIM: 244160131 Nama: Devi Kelas: A IPK: 3.72
NIM: 244160221 Nama: Badar Kelas: B IPK: 3.85

PreOrder Traversal:
NIM: 244160121 Nama: Ali Kelas: A IPK: 3.57
NIM: 244160185 Nama: Candra Kelas: C IPK: 3.21
NIM: 244160220 Nama: Dewi Kelas: B IPK: 3.54
NIM: 244160205 Nama: Ehsan Kelas: D IPK: 3.37
NIM: 244160170 Nama: Fizi Kelas: B IPK: 3.46
NIM: 244160221 Nama: Badar Kelas: B IPK: 3.85
NIM: 244160131 Nama: Devi Kelas: A IPK: 3.72

PostOrder Traversal:
NIM: 244160170 Nama: Fizi Kelas: B IPK: 3.46
NIM: 244160205 Nama: Ehsan Kelas: D IPK: 3.37
NIM: 244160220 Nama: Dewi Kelas: B IPK: 3.54
NIM: 244160185 Nama: Candra Kelas: C IPK: 3.21
NIM: 244160131 Nama: Devi Kelas: A IPK: 3.72
NIM: 244160221 Nama: Badar Kelas: B IPK: 3.85
NIM: 244160121 Nama: Ali Kelas: A IPK: 3.57

Penghapusan data mahasiswa
Jika 2 anak, current =
NIM: 244160131 Nama: Devi Kelas: A IPK: 3.72

Daftar semua mahasiswa setelah penghapusan 1 mahasiswa (in order traversal):
NIM: 244160185 Nama: Candra Kelas: C IPK: 3.21
NIM: 244160205 Nama: Ehsan Kelas: D IPK: 3.37
NIM: 244160170 Nama: Fizi Kelas: B IPK: 3.46
NIM: 244160220 Nama: Dewi Kelas: B IPK: 3.54
NIM: 244160131 Nama: Devi Kelas: A IPK: 3.72
NIM: 244160221 Nama: Badar Kelas: B IPK: 3.85

```

14.2.3 Pertanyaan!

1. Mengapa dalam binary search tree proses pencarian data bisa lebih efektif dilakukan dibanding binary tree biasa?

Jawaban:

Karena pada Binary Search Tree (BST) data tersusun terurut:

- a. nilai lebih kecil di kiri,
- b. nilai lebih besar di kanan.

Saat mencari data, program cukup memilih arah kiri atau kanan sehingga pencarian lebih cepat dibanding binary tree biasa yang harus mengecek semua node.

2. Untuk apakah di class Node, kegunaan dari atribut left dan right?

Jawaban:

- a. left digunakan untuk menyimpan referensi anak kiri.
- b. right digunakan untuk menyimpan referensi anak kanan.

Keduanya berfungsi menghubungkan node-node dalam tree.

3. a. Untuk apakah kegunaan dari atribut root di dalam class BinaryTree?

Jawaban:

root adalah node utama atau akar tree yang menjadi titik awal untuk semua proses seperti tambah, cari, traversal, dan hapus data.

- b. Ketika objek tree pertama kali dibuat, apakah nilai dari root?

Jawaban:

Nilai root adalah null karena tree masih kosong dan belum memiliki node.

4. Ketika tree masih kosong, dan akan ditambahkan sebuah node baru, proses apa yang akan terjadi?

Jawaban:

Node baru langsung dijadikan root karena belum ada node lain di dalam tree.

5. Perhatikan method add(), jelaskan secara detail untuk apa baris program tersebut!

```
parent = current;
if (mahasiswa.ipk < current.mahasiswa.ipk)
{
    current = current.left;
    if (current == null) {
        parent.left = newNode;
        return;
    }
}
else {
    current = current.right;
    if (current == null) {
        parent.right = newNode;
        return;
    }
}
```

Jawaban:

Baris program tersebut digunakan untuk menentukan posisi yang tepat saat menambahkan node baru ke dalam Binary Search Tree (BST). Program akan

membandingkan nilai IPK dari data baru dengan node yang sedang diperiksa (current). Jika IPK lebih kecil, maka program bergerak ke subtree kiri, sedangkan jika lebih besar atau sama maka program bergerak ke subtree kanan. Variabel parent digunakan untuk menyimpan node sebelumnya agar node baru nantinya dapat dihubungkan ke parent yang benar. Ketika posisi kiri atau kanan yang dituju bernilai null, berarti posisi kosong telah ditemukan dan node baru akan ditempatkan di sana. Dengan proses ini, susunan data dalam BST tetap teratur sesuai aturan binary search tree.

6. Jelaskan langkah-langkah pada method delete() saat menghapus sebuah node yang memiliki dua anak. Bagaimana method getSuccessor() membantu dalam proses ini?

Jawaban:

Langkah-langkahnya:

- a. Cari node yang akan dihapus.
- b. Cari successor menggunakan getSuccessor().
- c. Successor adalah node terkecil di subtree kanan.
- d. Successor menggantikan node yang dihapus.
- e. Hubungan node diperbaiki agar BST tetap valid.
- f. getSuccessor() membantu mencari pengganti yang tepat supaya struktur BST tetap teratur setelah penghapusan node.

14.3 Percobaan 2: Implementasi Binary Tree dengan Array

14.3.1 Langkah – langkah percobaan

Class BinaryTreeArray:

```
package Jobsheet14;
```

```
public class BinaryTreeArray10 {
```

```
    Mahasiswa10[] data;
```

```
    int idxLast;
```

```
    void populateData(Mahasiswa10[] data, int idxLast) {
```

```
        this.data = data;
```

```
        this.idxLast = idxLast;
```

```
    }
```

```
    void traverseInOrder(int idxStart) {
```

```

    if (idxStart <= idxLast) {
        if (data[idxStart] != null) {
            traverseInOrder(2 * idxStart + 1);

            data[idxStart].tampilInformasi();

            traverseInOrder(2 * idxStart + 2);
        }
    }
}
}
}

```

Class BinaryTreeArrayMain:

```

package Jobsheet14;

public class BinaryTreeArrayMain {
    public static void main(String[] args) {

        BinaryTreeArray10 bta = new BinaryTreeArray10();

        Mahasiswa10 m1 = new Mahasiswa10("244160121", "Ali", "A", 3.57);
        Mahasiswa10 m2 = new Mahasiswa10("244160185", "Candra", "C",
3.41);
        Mahasiswa10 m3 = new Mahasiswa10("244160221", "Badar", "B",
3.75);
        Mahasiswa10 m4 = new Mahasiswa10("244160220", "Dewi", "B", 3.35);
        Mahasiswa10 m5 = new Mahasiswa10("244160131", "Devi", "A", 3.48);
        Mahasiswa10 m6 = new Mahasiswa10("244160205", "Ehsan", "D",
3.61);
        Mahasiswa10 m7 = new Mahasiswa10("244160170", "Fizi", "B", 3.86);

        Mahasiswa10[] dataMahasiswa = {
            m1, m2, m3, m4, m5, m6, m7

```

```

    };

    int idxLast = 6;

    bta.populateData(dataMahasiswa, idxLast);

    System.out.println("\nInOrder Traversal Binary Tree:");
    bta.traverseInOrder(0);
}
}

```

14.3.2 Verifikasi Hasil Percobaan

```

InOrder Traversal Binary Tree:
NIM: 244160220 Nama: Dewi Kelas: B IPK: 3.35
NIM: 244160185 Nama: Candra Kelas: C IPK: 3.41
NIM: 244160131 Nama: Devi Kelas: A IPK: 3.48
NIM: 244160121 Nama: Ali Kelas: A IPK: 3.57
NIM: 244160205 Nama: Ehsan Kelas: D IPK: 3.61
NIM: 244160221 Nama: Badar Kelas: B IPK: 3.75
NIM: 244160170 Nama: Fizi Kelas: B IPK: 3.86

```

14.3.3 Pertanyaan!

1. Apakah kegunaan dari atribut data dan idxLast yang ada di class BinaryTreeArray?
Jawaban:
Atribut data digunakan untuk menyimpan seluruh node binary tree dalam bentuk array. Sedangkan idxLast digunakan untuk menyimpan indeks terakhir yang terisi data sehingga program mengetahui batas akhir data pada array.
2. Apakah kegunaan dari method populateData()?
Jawaban:
Method populateData() digunakan untuk mengisi data binary tree ke dalam array serta menyimpan indeks terakhir data yang digunakan.
3. Apakah kegunaan dari method traverseInOrder()?
Jawaban:
Method traverseInOrder() digunakan untuk menampilkan isi binary tree dengan urutan traversal inorder, yaitu mengunjungi subtree kiri, root, lalu subtree kanan.

4. Jika suatu node binary tree disimpan dalam array indeks 2, maka di indeks berapakah posisi left child dan right child masing-masing?

Jawaban:

Pada representasi binary tree menggunakan array:

- left child = $2 * \text{indeks} + 1$
- right child = $2 * \text{indeks} + 2$

Jika node berada pada indeks 2:

- left child berada di indeks 5
- right child berada di indeks 6

5. Apa kegunaan statement `int idxLast = 6` pada praktikum 2 percobaan nomor 4?

Jawaban:

Statement `int idxLast = 6` digunakan untuk menandai bahwa indeks terakhir yang terisi data pada array adalah indeks ke-6. Karena array berisi 7 data, maka indeks terakhirnya adalah 6.

6. Mengapa indeks $2idxStart+1$ dan $2idxStart+2$ digunakan dalam pemanggilan rekursif, dan apa kaitannya dengan struktur pohon biner yang disusun dalam array?

Jawaban:

Karena pada representasi binary tree menggunakan array terdapat aturan:

- child kiri berada pada indeks $2 * \text{parent} + 1$
- child kanan berada pada indeks $2 * \text{parent} + 2$

14.4 TUGAS PRAKTIKUM

14.4.1 Langkah-Langkah:

1. Buat method di dalam class `BinaryTree00` yang akan menambahkan node dengan cara rekursif (`addRekursif()`).
2. Buat method di dalam class `BinaryTree00` untuk menampilkan data mahasiswa dengan IPK paling kecil dan IPK yang paling besar (`cariMinIPK()` dan `cariMaxIPK()`) yang ada di dalam binary search tree.
3. Buat method dalam class `BinaryTree00` untuk menampilkan data mahasiswa dengan IPK di atas suatu batas tertentu, misal di atas 3.50 (`tampilMahasiswaIPKdiAtas(double ipkBatas)`) yang ada di dalam binary search tree.
4. Modifikasi class `BinaryTreeArray00` di atas, dan tambahkan :
 - method `add(Mahasiswa data)` untuk memasukkan data ke dalam binary tree
 - method `traversePreOrder()`

Jawaban:

Class `BinaryTree10`:

```
package Jobsheet14;
```

```
public class BinaryTree10 {  
    Node10 root;
```

```

public BinaryTree10() {
    root = null;
}

public boolean isEmpty() {
    return root == null;
}

public void add(Mahasiswa10 mahasiswa) {
    Node10 newNode = new Node10(mahasiswa);
    if (isEmpty()) {
        root = newNode;
    } else {
        Node10 current = root;
        Node10 parent = null;
        while (true) {
            parent = current;
            if (mahasiswa.ipk < current.mahasiswa.ipk) {
                current = current.left;
                if (current == null) {
                    parent.left = newNode;
                    return;
                }
            } else {
                current = current.right;
                if (current == null) {
                    parent.right = newNode;
                    return;
                }
            }
        }
    }
}

public boolean find(double ipk) {
    boolean result = false;
    Node10 current = root;
    while (current != null) {
        if (current.mahasiswa.ipk == ipk) {
            result = true;
            break;
        } else if (ipk > current.mahasiswa.ipk) {
            current = current.right;
        } else {
            current = current.left;
        }
    }
    return result;
}

```

```

public void traversePreOrder(Node10 node) {
    if (node != null) {
        node.mahasiswa.tampilInformasi();
        traversePreOrder(node.left);
        traversePreOrder(node.right);
    }
}

public void traverseInOrder(Node10 node) {
    if (node != null) {
        traverseInOrder(node.left);
        node.mahasiswa.tampilInformasi();
        traverseInOrder(node.right);
    }
}

public void traversePostOrder(Node10 node) {
    if (node != null) {
        traversePostOrder(node.left);
        traversePostOrder(node.right);
        node.mahasiswa.tampilInformasi();
    }
}

public Node10 getSuccessor (Node10 del) {
    Node10 successor = del.right;
    Node10 successorParent = del;
    while (successor.left != null) {
        successorParent = successor;
        successor = successor.left;
    }
    if (successor != del.right) {
        successorParent.left = successor.right;
        successor.right = del.right;
    }
    return successor;
}

public void delete (double ipk) {
    if (isEmpty()) {
        System.out.println("Binary tree kosong");
        return;
    }
    // cari node (current) yang akan dihapus
    Node10 parent = root;
    Node10 current = root;
    boolean isLeftChild = false;
    while (current != null) {
        if (current.mahasiswa.ipk == ipk) {
            break;

```

```

    } else if (ipk < current.mahasiswa.ipk) {
        parent = current;
        current = current.left;
        isLeftChild = true;
    } else if (ipk > current.mahasiswa.ipk) {
        parent = current;
        current = current.right;
        isLeftChild = false;
    }
}

// penghapusan
if (current == null) {
    System.out.println("Data tidak ditemukan");
    return;
} else {
    // jika tidak ada anak (leaf), maka node dihapus
    if (current.left == null && current.right == null) {
        if (current == root) {
            root = null;
        } else {
            if (isLeftChild) {
                parent.left = null;
            } else {
                parent.right = null;
            }
        }
    }
    } else if (current.left == null) { // jika hanya punya 1 anak (kanan)
        if (current == root) {
            root = current.right;
        } else {
            if (isLeftChild) {
                parent.left = current.right;
            } else {
                parent.right = current.right;
            }
        }
    }
    } else if (current.right == null) { // jika hanya punya 1 anak (kiri)
        if (current == root) {
            root = current.left;
        } else {
            if (isLeftChild) {
                parent.left = current.left;
            } else {
                parent.right = current.left;
            }
        }
    }
    } else { // jika punya 2 anak
        Node10 successor = getSuccessor(current);
        System.out.println("Jika 2 anak, current = ");
    }
}

```

```

        successor.mahasiswa.tampilInformasi();
        if (current == root) {
            root = successor;
        } else {
            if (isLeftChild) {
                parent.left = successor;
            } else {
                parent.right = successor;
            }
        }
        successor.left = current.left;
    }
}

public void addRekursif(Mahasiswa10 mahasiswa) {
    root = addRekursif(root, mahasiswa);
}

public Node10 addRekursif(Node10 current, Mahasiswa10 mahasiswa) {
    if (current == null) {
        return new Node10(mahasiswa);
    }

    if (mahasiswa.ipk < current.mahasiswa.ipk) {
        current.left = addRekursif(current.left, mahasiswa);
    } else if (mahasiswa.ipk > current.mahasiswa.ipk) {
        current.right = addRekursif(current.right, mahasiswa);
    }
    return current;
}

public void cariMinIPK() {
    if (root == null) {
        System.out.println("Tree kosong");
        return;
    }

    Node10 current = root;

    while (current.left != null) {
        current = current.left;
    }

    current.mahasiswa.tampilInformasi();
}

public void cariMaxIPK() {
    if (root == null) {
        System.out.println("Tree kosong");
    }
}

```

```

        return;
    }

    Node10 current = root;

    while (current.right != null) {
        current = current.right;
    }

    current.mahasiswa.tampilInformasi();
}

public void tampilMahasiswaIPKdiAtas(double ipkBatas) {
    tampilMahasiswaIPKdiAtas(root, ipkBatas);
}

public void tampilMahasiswaIPKdiAtas(Node10 node, double ipkBatas) {
    if (node != null) {
        tampilMahasiswaIPKdiAtas(node.left, ipkBatas);

        if (node.mahasiswa.ipk > ipkBatas) {
            node.mahasiswa.tampilInformasi();
        }

        tampilMahasiswaIPKdiAtas(node.right, ipkBatas);
    }
}
}

```

Class BinaryTreeMain10:

```

package Jobsheet14;

public class BinaryTreeMain10 {

    public static void main(String[] args) {
        BinaryTree10 bst = new BinaryTree10();

        bst.add(new Mahasiswa10("244160121", "Ali", "A", 3.57));
        bst.add(new Mahasiswa10("244160221", "Badar", "B", 3.85));
        bst.add(new Mahasiswa10("244160185", "Candra", "C", 3.21));
        bst.add(new Mahasiswa10("244160220", "Dewi", "B", 3.54));

        System.out.println("\nDaftar semua mahasiswa (in order traversal):");
        bst.traverseInOrder(bst.root);

        System.out.println("\nPencarian data mahasiswa:");
        System.out.print("Cari mahasiswa dengan ipk: 3.54 : ");
        String hasilCari = bst.find(3.54) ? "Ditemukan" : "Tidak ditemukan";
        System.out.println(hasilCari);
    }
}

```

```

System.out.print("Cari mahasiswa dengan ipk: 3.22 : ");
hasilCari = bst.find(3.22) ? "Ditemukan" : "Tidak ditemukan";
System.out.println(hasilCari);

bst.add(new Mahasiswa10("244160131", "Devi", "A", 3.72));
bst.add(new Mahasiswa10("244160205", "Ehsan", "D", 3.37));
bst.add(new Mahasiswa10("244160170", "Fizi", "B", 3.46));

System.out.println("\nDaftar semua mahasiswa setelah penambahan 3
mahasiswa:");
System.out.println("InOrder Traversal:");
bst.traverseInOrder(bst.root);

System.out.println("\nPreOrder Traversal:");
bst.traversePreOrder(bst.root);

System.out.println("\nPostOrder Traversal:");
bst.traversePostOrder(bst.root);

System.out.println("\nPenghapusan data mahasiswa");
bst.delete(3.57);

System.out.println("\nDaftar semua mahasiswa setelah penghapusan 1
mahasiswa (in order traversal):");
bst.traverseInOrder(bst.root);

// tugas praktikum
System.out.println("\nMenambahkan data secara rekursif:");
bst.addRekursif(new Mahasiswa10("244160111", "Galih", "TI-1G",
3.90));

System.out.println("\nData setelah addRekursif():");
bst.traverseInOrder(bst.root);

System.out.println("\nMahasiswa dengan IPK terkecil:");
bst.cariMinIPK();

System.out.println("\nMahasiswa dengan IPK terbesar:");
bst.cariMaxIPK();

System.out.println("\nMahasiswa dengan IPK di atas 3.50:");
bst.tampilMahasiswaIPKdiAtas(3.50);

System.out.println("\nPenghapusan data mahasiswa");
bst.delete(3.46);

System.out.println("\nDaftar semua mahasiswa setelah penghapusan 1
mahasiswa:");
bst.traverseInOrder(bst.root);

```

```
}  
}
```

Class BinaryTreeArray10:

```
package Jobsheet14;  
  
public class BinaryTreeArray10 {  
    Mahasiswa10[] data;  
    int idxLast;  
  
    public BinaryTreeArray10(int size) {  
        data = new Mahasiswa10[size];  
        idxLast = -1;  
    }  
  
    void populateData(Mahasiswa10[] data, int idxLast) {  
        this.data = data;  
        this.idxLast = idxLast;  
    }  
  
    void traverseInOrder(int idxStart) {  
        if (idxStart <= idxLast) {  
            if (data[idxStart] != null) {  
                traverseInOrder(2 * idxStart + 1);  
  
                data[idxStart].tampilInformasi();  
  
                traverseInOrder(2 * idxStart + 2);  
            }  
        }  
    }  
  
    void traversePreOrder(int idxStart) {  
        if (idxStart <= idxLast) {  
            if (data[idxStart] != null) {  
                data[idxStart].tampilInformasi();  
                traversePreOrder(2 * idxStart + 1);  
                traversePreOrder(2 * idxStart + 2);  
            }  
        }  
    }  
  
    void add(Mahasiswa10 data) {  
        this.data[++idxLast] = data;  
    }  
}
```

Class BinaryTreeArrayMain10:

```

package Jobsheet14;

import java.util.Scanner;

public class BinaryTreeArrayMain {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Masukkan size array: ");
        int sizeArr = sc.nextInt();
        BinaryTreeArray10 bta = new BinaryTreeArray10(sizeArr);

        Mahasiswa10 m1 = new Mahasiswa10("244160121", "Ali", "A", 3.57);
        Mahasiswa10 m2 = new Mahasiswa10("244160185", "Candra", "C",
3.41);
        Mahasiswa10 m3 = new Mahasiswa10("244160221", "Badar", "B",
3.75);
        Mahasiswa10 m4 = new Mahasiswa10("244160220", "Dewi", "B", 3.35);
        Mahasiswa10 m5 = new Mahasiswa10("244160131", "Devi", "A", 3.48);
        Mahasiswa10 m6 = new Mahasiswa10("244160205", "Ehsan", "D",
3.61);
        Mahasiswa10 m7 = new Mahasiswa10("244160170", "Fizi", "B", 3.86);

        bta.add(m1);
        bta.add(m2);
        bta.add(m3);
        bta.add(m4);
        bta.add(m5);
        bta.add(m6);
        bta.add(m7);

        System.out.println("\nInOrder Traversal Binary Tree:");
        bta.traverseInOrder(0);

        System.out.println("\nPreOrder Traversal Binary Tree:");
        bta.traversePreOrder(0);
        sc.close();
    }
}

```

14.4.2 Output:

No: 1,2 , 3

```
Daftar semua mahasiswa (in order traversal):
NIM: 244160185 Nama: Candra Kelas: C IPK: 3.21
NIM: 244160220 Nama: Dewi Kelas: B IPK: 3.54
NIM: 244160121 Nama: Ali Kelas: A IPK: 3.57
NIM: 244160221 Nama: Badar Kelas: B IPK: 3.85

Pencarian data mahasiswa:
Cari mahasiswa dengan ipk: 3.54 : Ditemukan
Cari mahasiswa dengan ipk: 3.22 : Tidak ditemukan

Daftar semua mahasiswa setelah penambahan 3 mahasiswa:
InOrder Traversal:
NIM: 244160185 Nama: Candra Kelas: C IPK: 3.21
NIM: 244160205 Nama: Ehsan Kelas: D IPK: 3.37
NIM: 244160170 Nama: Fizi Kelas: B IPK: 3.46
NIM: 244160220 Nama: Dewi Kelas: B IPK: 3.54
NIM: 244160121 Nama: Ali Kelas: A IPK: 3.57
NIM: 244160131 Nama: Devi Kelas: A IPK: 3.72
NIM: 244160221 Nama: Badar Kelas: B IPK: 3.85

PreOrder Traversal:
NIM: 244160121 Nama: Ali Kelas: A IPK: 3.57
NIM: 244160185 Nama: Candra Kelas: C IPK: 3.21
NIM: 244160220 Nama: Dewi Kelas: B IPK: 3.54
NIM: 244160205 Nama: Ehsan Kelas: D IPK: 3.37
NIM: 244160170 Nama: Fizi Kelas: B IPK: 3.46
NIM: 244160221 Nama: Badar Kelas: B IPK: 3.85
NIM: 244160131 Nama: Devi Kelas: A IPK: 3.72

PostOrder Traversal:
NIM: 244160170 Nama: Fizi Kelas: B IPK: 3.46
NIM: 244160205 Nama: Ehsan Kelas: D IPK: 3.37
NIM: 244160220 Nama: Dewi Kelas: B IPK: 3.54
NIM: 244160185 Nama: Candra Kelas: C IPK: 3.21
NIM: 244160131 Nama: Devi Kelas: A IPK: 3.72
NIM: 244160221 Nama: Badar Kelas: B IPK: 3.85
NIM: 244160121 Nama: Ali Kelas: A IPK: 3.57
```

```
Penghapusan data mahasiswa
Jika 2 anak, current -
NIM: 244160131 Nama: Devi Kelas: A IPK: 3.72

Daftar semua mahasiswa setelah penghapusan 1 mahasiswa (in order traversal):
NIM: 244160185 Nama: Candra Kelas: C IPK: 3.21
NIM: 244160205 Nama: Ehsan Kelas: D IPK: 3.37
NIM: 244160170 Nama: Fizi Kelas: B IPK: 3.46
NIM: 244160220 Nama: Dewi Kelas: B IPK: 3.54
NIM: 244160131 Nama: Devi Kelas: A IPK: 3.72
NIM: 244160221 Nama: Badar Kelas: B IPK: 3.85

Menambahkan data secara rekursif:

Data setelah addRekursif():
NIM: 244160185 Nama: Candra Kelas: C IPK: 3.21
NIM: 244160205 Nama: Ehsan Kelas: D IPK: 3.37
NIM: 244160170 Nama: Fizi Kelas: B IPK: 3.46
NIM: 244160220 Nama: Dewi Kelas: B IPK: 3.54
NIM: 244160131 Nama: Devi Kelas: A IPK: 3.72
NIM: 244160221 Nama: Badar Kelas: B IPK: 3.85
NIM: 244160111 Nama: Galih Kelas: TI-1G IPK: 3.9

Mahasiswa dengan IPK terkecil:
NIM: 244160185 Nama: Candra Kelas: C IPK: 3.21

Mahasiswa dengan IPK terbesar:
NIM: 244160111 Nama: Galih Kelas: TI-1G IPK: 3.9

Mahasiswa dengan IPK di atas 3.50:
NIM: 244160220 Nama: Dewi Kelas: B IPK: 3.54
NIM: 244160131 Nama: Devi Kelas: A IPK: 3.72
NIM: 244160221 Nama: Badar Kelas: B IPK: 3.85
NIM: 244160111 Nama: Galih Kelas: TI-1G IPK: 3.9

Penghapusan data mahasiswa

Daftar semua mahasiswa setelah penghapusan 1 mahasiswa:
NIM: 244160185 Nama: Candra Kelas: C IPK: 3.21
NIM: 244160205 Nama: Ehsan Kelas: D IPK: 3.37
NIM: 244160220 Nama: Dewi Kelas: B IPK: 3.54
NIM: 244160131 Nama: Devi Kelas: A IPK: 3.72
NIM: 244160221 Nama: Badar Kelas: B IPK: 3.85
NIM: 244160111 Nama: Galih Kelas: TI-1G IPK: 3.9
```

No 4:

```
Masukkan size array: 7
```

```
InOrder Traversal Binary Tree:
```

```
NIM: 244160220 Nama: Dewi Kelas: B IPK: 3.35  
NIM: 244160185 Nama: Candra Kelas: C IPK: 3.41  
NIM: 244160131 Nama: Devi Kelas: A IPK: 3.48  
NIM: 244160121 Nama: Ali Kelas: A IPK: 3.57  
NIM: 244160205 Nama: Ehsan Kelas: D IPK: 3.61  
NIM: 244160221 Nama: Badar Kelas: B IPK: 3.75  
NIM: 244160170 Nama: Fizi Kelas: B IPK: 3.86
```

```
PreOrder Traversal Binary Tree:
```

```
NIM: 244160121 Nama: Ali Kelas: A IPK: 3.57  
NIM: 244160185 Nama: Candra Kelas: C IPK: 3.41  
NIM: 244160220 Nama: Dewi Kelas: B IPK: 3.35  
NIM: 244160131 Nama: Devi Kelas: A IPK: 3.48  
NIM: 244160221 Nama: Badar Kelas: B IPK: 3.75  
NIM: 244160205 Nama: Ehsan Kelas: D IPK: 3.61  
NIM: 244160170 Nama: Fizi Kelas: B IPK: 3.86
```